GenAI Usage Reflection

Sheheryar Azam

Tool used

The only GenAI tool I used was Claude (Anthropic), through its chat interface.

How I used it

My pattern was to ask it to draft or explain something, then question it, run it, and correct it against the actual requirements, my teammates' documents, and the live database. Many of my prompts were "why" questions, because I wanted to be able to explain every decision.

Task 2: Functional dependencies and BCNF

- What I asked: to help structure the FD list and BCNF justification for each relation, and to sanity-check whether particular dependencies actually held.
- What I rejected (the key decisions): The tool proposed several FD's that I chose to reject. The first was Doctor ID --> clinic ID in Appointment, which would mean that knowing the doctor on an appointment would be enough to know the clinic, where actually on an appointment the clinic records where the booking was held, which is separate from the doctor's employing clinic which is stored in Staff. A doctor is employed at one clinic at a time, but that can change and the doctor can hold future appointments at different clinics. So, the table would store different clinics for a doctor that would fail that FD. And had I accepted it, would create a transitive dependency of appointment ID --> Doctor ID --> clinic ID which would break the BCNF. Secondly it proposed approvingReceptionistID --> clinicID in ClinicService. This would mean knowing which receptionist approved of a clinic service offering is enough to know the clinic i.e. that each receptionist only ever approves offerings for one clinic. The scenario states no such restriction, so the approving receptionist doesn't determine the clinic
- **How I validated:** I cross-checked every FD against Johns EER and Yoon's relational mapping so all three documents agreed, and against the textbook BCNF rule (every determinant must be a candidate key).

How my judgement contributed

The tool was fastest at producing first drafts and at explaining concepts on demand. The decisions that actually shaped the final solution, which FDs to reject and why were my

decisions.

Yoon Seong Lee

Tools used

Claude (Anthropic) and ChatGPT, used in parallel rather than relying on either one alone.

How I used them

For my parts — assumptions, documentation consistency, the stored procedure logic, and the ORM booking DAO logic — I ran the same material through both tools independently and compared where they agreed and where they diverged. I treated each output as a suggestion to be checked, not as a conclusion.

What I caught and decided

The most useful cases were where the two tools disagreed. One example was the unique constraint on medicareOrInsuranceNo. The tools focused on different layers: one on the ORM entity annotations, the other on the database schema in createTables.sql. Neither answer was complete on its own.

I decided that the database should be the authoritative enforcement layer, because uniqueness is a core data integrity rule and should not depend only on application-side validation. ORM annotations can serve as an additional validation layer, but they should not replace or contradict the database constraint. This also clarified the responsibility boundary between layers: the constraint belonged in createTables.sql, while the ORM code had to stay consistent with that schema.

A smaller but important case was the bookingMethod value. Some AI-generated suggestions used "inperson", which reads naturally but does not match the database CHECK constraint. The database requires 'in person' with a space, and the allowed values are online, phone, and in person. A mismatch like this can look fine in code but fail at runtime, so I cross-checked generated values against the actual CHECK constraints instead of trusting the output.

How I validated the outputs

I checked every suggestion against the implemented design rather than treating all AI feedback as equally applicable. For each one I asked two things: does it match the actual schema, SQL scripts, and CHECK constraints, and which layer does it belong to — the database, the ORM, or the documentation. If a suggestion did not match the implemented database design, I did not accept it.

How my judgement contributed

AI output is often surface-plausible: it can look correct and read well, but it may not know the final implementation details or which teammate owns which layer. Running two tools against each other helped expose this, but it also added work — comparing two sets of suggestions and reconciling them took more time than a single tool would have.

The cross-validation was useful precisely because the tools sometimes disagreed, but it never removed the need for human review. The actual decisions came from manual checking, not from either tool's output alone.